Empirical Evaluation of an Approximation Algorithm for Bipartite Matching

Rishabh Jain

April 26, 2026

1 Introduction

Maximum bipartite matching is a fundamental problem in combinatorial optimization with applications in scheduling, resource allocation, and network design. Given a bipartite graph $G = (U, V, E)$, the goal is to find the largest set of edges such that no two edges share an endpoint. Although Classical algorithms such as Hopcroft–Karp compute an exact maximum matching, it is costly on large graphs, motivating the study of faster approximation algorithms.

The simplest approximation approach is greedy maximal matching. Greedy scans the edges and adds an edge whenever both endpoints are unmatched. It runs in linear time and always returns a 1/2-approximation to the maximum matching. Recent algorithms obtain approximation ratios strictly larger than 1/2 by combining randomized sparsification, local matching oracles, and structural decompositions of the residual graph. In this project, we implement and evaluate one such modern approximation algorithm for bipartite matching specifically.

2 Algorithmic Background

2.1 Bipartite Matching

Let $G = (U, V, E)$ be a bipartite graph, where U and V are disjoint vertex sets and $E \subseteq U \times V$ is the set of edges. A *matching* $M \subseteq E$ is a set of edges such that no two edges in M share an endpoint. The goal is to compute a matching of maximum cardinality. We denote the size of a maximum matching by OPT.

For a matching M, let $V(M)$ denote the set of all vertices incident to edges in M. Since the graph is bipartite, the full vertex set is $U \cup V$, and the unmatched vertices are $(U \cup V) \setminus V(M)$.

2.2 Baseline Algorithms

Greedy Matching. The greedy algorithm constructs a maximal matching by scanning edges in some order and adding an edge (u, v) whenever both endpoints are currently unmatched. This algorithm runs in $O(m)$ time and returns a matching M_{greedy} satisfying

$$|M_{\text{greedy}}| \geq \frac{1}{2} \cdot \text{OPT}.$$

Hopcroft–Karp Algorithm. The Hopcroft–Karp algorithm computes a maximum bipartite matching exactly using repeated searches for shortest augmenting paths. It runs in $O(m\sqrt{n})$ time, where $n = |U| + |V|$ and $m = |E|$. In our experiments, we use Hopcroft–Karp to compute the ground-truth value OPT against which approximation ratios are measured.

2.3 Modern Approximation Framework

The modern algorithm we study aims to improve on the greedy matching by combining randomized sparsification with two complementary estimators. The algorithm first constructs a partial matching M and then estimates additional matching structure through two cases.

Sparsification. This constructs a partial matching M using randomized neighbor sampling. For each currently unmatched vertex, the algorithm samples a prescribed number of incident edges and adds an edge to M if both endpoints are unmatched. The number of samples we get is

$$c = \lceil 2\sqrt{n} \log n \rceil .$$

Case 1: Residual Matching Structure. After sparsification, the algorithm considers the subgraph induced by the unmatched vertices,

$$G[(U \cup V) \setminus V(M)].$$

Within this residual, the algorithm uses a random greedy maximal matching oracle to estimate the size of an additional matching M'. It then constructs a Case 1 copied graph whose copy counts depend on whether vertices are matched by this residual matching oracle. A second random greedy oracle is used on this copied graph to estimate a quantity denoted B_1. These estimates are combined as

$$\mu_1 = |M| + \left(1 - \frac{1}{b}\right) \widehat{|M'|} + \frac{1}{kb} \widehat{|B_1|},$$

where $b = 1 + \sqrt{2}$ and k is a sufficiently large capacity parameter.

Case 2: Matched–Unmatched Structure. Case 2 constructs a different copied graph based directly on the sparsification matching M. Vertices matched by M receive k copies, while unmatched vertices receive approximately kb copies. Edges in the copied graph encode interactions between matched and unmatched vertices in the original graph. A random greedy maximal matching oracle is then used to estimate a quantity B_2, yielding

$$\mu_2 = \left(1 - \frac{1}{b}\right) |M| + \frac{1}{kb} \widehat{|B_2|}.$$

Final Estimate. The algorithm outputs

$$\max(\mu_1, \mu_2)$$

2.4 Empirical Considerations

Although the theoretical guarantee is worst-case, the practical behavior of the algorithm depends strongly on the input graph structure. In particular, Case 1 is useful only when the sparsification matching leaves a residual graph with nontrivial matching structure. If the initial matching M covers most vertices, then the residual graph is small or empty, and the Case 1 contribution may be negligible. In those settings, the final estimate is typically dominated by the sparsification matching and the Case 2 estimator.

Understanding when each component contributes meaningfully is a central goal of the experimental evaluation.

3 Experimental Setup

3.1 Graph Families

We evaluate the algorithms on three families of bipartite graphs. All graph families are balanced, with $|U| = |V| = n$, so the total number of vertices is $2n$. These graph families were chosen to represent different structural regimes: uniformly random graphs, graphs with highly skewed degree distributions, and graphs with controlled near-regular degrees.

Erdős–Rényi bipartite graphs. The first graph family is the bipartite Erdős–Rényi model. For each pair $(u, v) \in U \times V$, the edge is included independently with probability p. Thus, the expected number of edges is pn^2, and the expected degree of each vertex is pn. This family provides a standard random-graph baseline in which degrees are relatively homogeneous when pn is moderately large. We vary p to study how the algorithms behave as the graph transitions from sparse to denser regimes.

Power-law bipartite graphs. The second graph family consists of bipartite graphs with power-law-like degree skew. These graphs are generated by sampling left and right endpoints according to rank-based weights, so a small number of vertices are likely to receive many edges while many vertices have low degree. The total number of edges is chosen to match the expected edge count of an Erdős–Rényi graph with the same density parameter.

Near-regular bipartite graphs. The third graph family consists of near-regular bipartite graphs with left degree approximately $\Theta(\sqrt{n})$. For each left vertex, we sample a fixed number of distinct right neighbors uniformly at random. This gives every left vertex exactly the target degree, while right degrees concentrate around the same value in expectation. These graphs provide a controlled setting in which the graph is neither extremely sparse nor highly skewed. They are useful for evaluating how the algorithms behave when local degrees are relatively balanced and the graph has enough density to support large matchings.

3.2 Parameters and Trials

For each graph family, the experiments are divided into two tiers. The first tier is the main algorithm comparison, while the second tier studies the sensitivity of the modern algorithm to the theorem parameter ϵ and the derived copied-graph capacity parameter k.

In the main comparison, we use

$$n \in \{50, 100, 250, 500, 1000\}.$$

For Erdős–Rényi and power-law graphs, we parameterize density by expected average degree rather than by a fixed edge probability. For a target expected average degree d, we set

$$p = \frac{d}{n}$$

for Erdős–Rényi graphs. This keeps the expected degree comparable as n varies. We use

$$d \in \{2, 4, 8, 16, 32\}.$$

For power-law graphs, we use the same values of d to determine the target number of edges, choosing approximately dn edges. This makes the power-law graphs comparable to Erdős–Rényi graphs at similar average degree while changing the degree distribution.

For near-regular graphs, the density parameter is not used directly. Instead, each left vertex is assigned

$$d = \lceil \sqrt{n} \rceil$$

distinct right neighbors sampled uniformly at random. This produces graphs with exactly nd edges and average degree approximately d on both sides.

The modern algorithm has a theorem parameter ϵ that determines the copied-graph capacity parameter k. When k is not specified explicitly, we use the paper-aligned choice

$$k = \left\lfloor \frac{1}{(1+\sqrt{2})\epsilon^3} \right\rfloor + 1.$$

We also run a separate epsilon sensitivity experiment to measure the cost of more theorem-aligned parameter choices. In this experiment, we use

$$n \in \{50, 100, 250\}, \qquad d \in \{4, 8, 16\},$$

with 10 trials per setting, and vary

$$\epsilon \in \{0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1\}.$$

The corresponding values of k increase rapidly as ϵ decreases. In particular, $\epsilon = 0.1$ gives a much larger copied-graph capacity than the values used in the main comparison. This experiment therefore measures the tradeoff between approximation behavior, copied-graph size, and runtime.

3.3 Metrics

We evaluate the algorithms using both external performance metrics and internal diagnostics of the modern estimator. The external metrics measure approximation quality and runtime, while the internal metrics help explain which components of the modern algorithm contribute to its final estimate.

Approximation ratio. For each graph instance, Hopcroft–Karp is used to compute the optimal matching size OPT. For an algorithm A that outputs a matching or estimate of size $A(G)$, we define its approximation ratio as

$$\frac{A(G)}{\mathrm{OPT}}.$$

For greedy, $A(G)$ is the size of the matching returned by the algorithm. For the modern algorithm, $A(G)$ is the final estimate $\max(\mu_1, \mu_2)$. Hopcroft–Karp has $A(G) = 1$ by definition.

Runtime. We measure wall-clock runtime for each algorithm on every graph instance. For greedy, this is the time required to construct the matching. For Hopcroft–Karp, it is the time required to compute the exact maximum matching. For the modern algorithm, we measure both total runtime and a phase-level breakdown:

$$t_{\text{sparsify}}, \qquad t_{\text{Case 1}}, \qquad t_{\text{Case 2}}, \qquad t_{\text{other}}.$$

The first three quantities correspond to the sparsification step, the Case 1 estimator, and the Case 2 estimator. The remaining time is recorded as overhead. This breakdown is useful because the modern algorithm performs several conceptually different operations, and the copied-graph oracle computations may dominate runtime for some parameter settings.

Modern estimator components. For the modern algorithm, we record the initial sparsification matching size $|M|$, the Case 1 estimate μ_1, the Case 2 estimate μ_2, and the selected case:

$$\arg\max\{\mu_1, \mu_2\}.$$

We also record the intermediate estimated quantities $\widehat{|M'|}$, $\widehat{|B_1|}$, and $\widehat{|B_2|}$. These values allow us to decompose the final estimate into the contributions

$$|M|, \qquad \left(1 - \frac{1}{b}\right)\widehat{|M'|}, \qquad \frac{1}{kb}\widehat{|B_1|}, \qquad \frac{1}{kb}\widehat{|B_2|},$$

where $b = 1 + \sqrt{2}$. This decomposition helps identify whether the estimate is primarily driven by the initial sparsification matching, the residual structure captured by Case 1, or the copied-graph structure captured by Case 2.

Copied-graph size and parameter sensitivity. For the modern algorithm, we record the effective value of k, the value of ϵ, the sparsification sample count, and the number of copied vertices in the auxiliary graph views. These quantities are especially important in the epsilon sensitivity experiment. Since

$$k = \left\lfloor \frac{1}{(1+\sqrt{2})\epsilon^3} \right\rfloor + 1,$$

smaller values of ϵ produce larger copied-graph capacities. Recording copied-graph size and runtime allows us to measure the practical cost of using more theorem-aligned parameter values.

3.4 Approximation Quality

Table 1 reports the mean approximation ratio by graph family. On Erdős–Rényi graphs, greedy and the modern algorithm perform almost identically, with mean approximation ratios of 0.9198 and 0.9195, respectively. On power-law graphs, all algorithms perform worse than on Erdős–Rényi graphs, reflecting the difficulty introduced by skewed degree distributions. On near-regular graphs, the modern algorithm performs best among the approximation algorithms, obtaining mean ratio 0.9556, compared with 0.9465 for greedy.

Graph family	Mean OPT	Greedy ratio	Modern ratio
Erdős–Rényi	361.666	0.9198	0.9195
Power-law	172.118	0.8072	0.7853
Near-regular	380.000	0.9465	0.9556

Table 1: Mean approximation quality by graph family

Figure 1 shows approximation ratio as a function of graph size. Each point averages over all Tier 1 trials and degree settings for the corresponding graph family and value of n. The horizontal Hopcroft–Karp line at 1 represents the optimum. On Erdős–Rényi graphs, greedy and the modern estimator remain very close across all graph sizes. On power-law graphs, the approximation ratios are lower overall, and the modern estimator remains below the greedy baselines across the tested sizes. On near-regular graphs, approximation quality improves with n, which is expected because the near-regular construction uses degree $\lceil \sqrt{n} \rceil$, so larger graphs in this family are also denser.

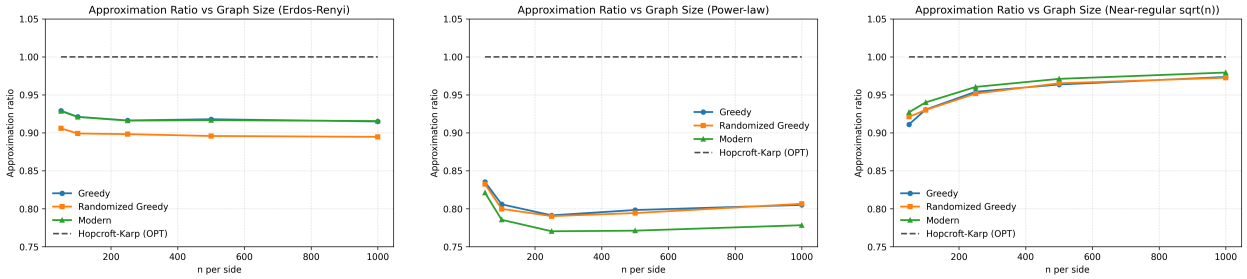

Figure 1: Approximation ratio versus graph sizes.

3.5 Runtime Scaling

We next compare the runtime of the algorithms. Figure 2 shows mean runtime as a function of graph size for each graph family. The y-axis is plotted on a logarithmic scale because the runtimes differ by several orders of magnitude. Greedy and randomized greedy are the fastest methods across all graph families, as expected from their linear-time scans over the edge list. Hopcroft–Karp is slower than greedy but remains very fast on the tested instances. The modern estimator is substantially slower than both baselines, primarily because it runs local random greedy matching oracles on auxiliary copied graph views.

On Erdős–Rényi graphs, the modern algorithm's mean runtime grows from 0.1301 seconds at $n = 50$ to 0.9932 seconds at $n = 1000$. On power-law graphs, the modern algorithm is even more significantly slower: its mean runtime grows from 0.2968 seconds at $n = 50$ to 12.4281 seconds at $n = 1000$. This is the largest runtime among the tested graph families. On near-regular graphs, the modern runtime grows from 0.1122 seconds at $n = 50$ to 1.8702 seconds at $n = 1000$. This does not contradict the theoretical motivation of the modern algorithm as it is designed around local oracle access and large auxiliary graph constructions. However, our benchmark compares concrete Python implementations on moderate-size explicit graphs. In this setting, the overhead of oracle simulation and copied-graph access dominates the runtime.

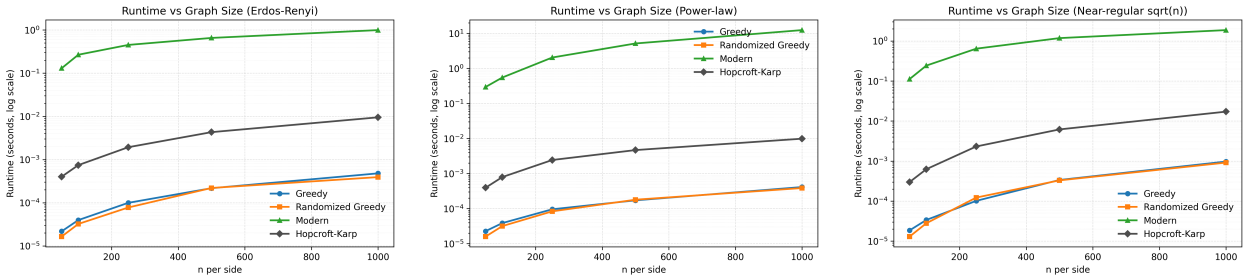

Figure 2: Runtime versus graph size

To understand the cost of the modern estimator, Figure 3 decomposes its runtime into sparsification, Case 1, Case 2, and other overhead. The breakdown shows that Case 2 dominates the runtime across all graph families. Averaged over all Tier 1 trials, the Erdős–Rényi instances spend 0.4854 of 0.4992 seconds in Case 2. The power-law instances spend 4.0503 of 4.0996 seconds in Case 2, and the near-regular instances spend 0.7991 of 0.8098 seconds in Case 2. Sparsification and Case 1 are comparatively small contributors to total runtime.

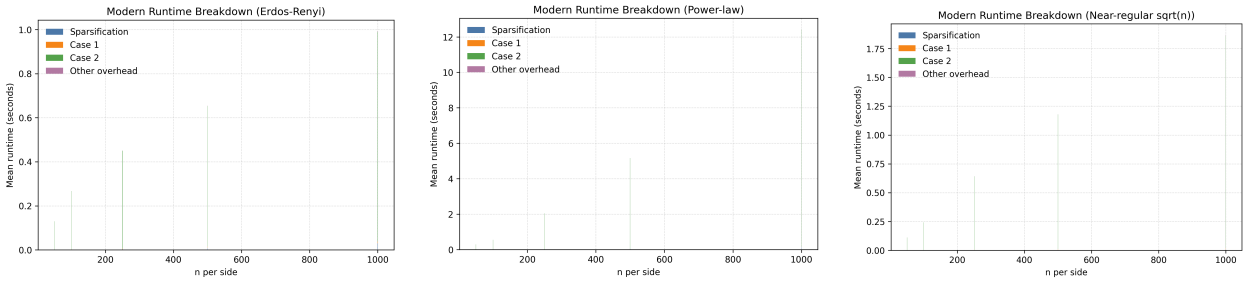

Figure 3: Runtime breakdown for the modern estimator

3.6 Internal Behavior of the Modern Algorithm

We next examine the internal behavior of the modern estimator. This is important because the final output does not by itself explain which part of the algorithm is responsible for the estimate. We therefore study three internal quantities: which case is selected, how the estimate decomposes into its main terms, and whether the estimator tends to overestimate or underestimate the optimum.

Figure 4 shows the fraction of trials in which the modern algorithm selected Case 1, meaning $\mu_1 \geq \mu_2$. On Erdős–Rényi and near-regular graphs, Case 1 is selected in 100% of Tier 1 trials. On power-law graphs, Case 1 is selected in 73.8% of trials. The power-law heatmap shows that Case 2 becomes more relevant for some higher-degree settings, especially when the graph has both skewed degree structure and enough edges for the copied Case 2 graph to contribute.

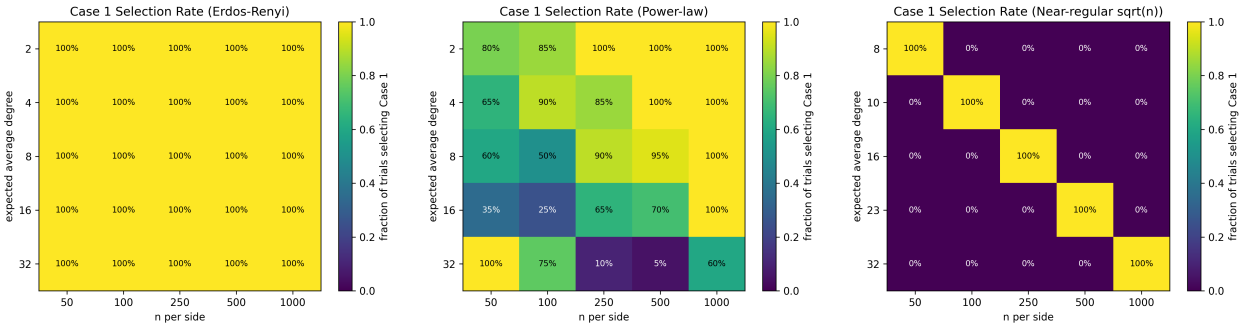

Figure 4: Fraction of trials in which the modern algorithm selected Case 1

However, selecting Case 1 does not necessarily mean that the residual Case 1 structure contributes substantially. In our experiments, the residual matching estimate $\widehat{|M'|}$ and the Case 1 copied-graph estimate $\widehat{|B_1|}$ are both zero on average for all three graph families. As a result, the Case 1 estimate usually reduces to

$$\mu_1 = |M|.$$

Thus, when Case 1 is selected on Erdős–Rényi and near-regular graphs, the selected estimate is primarily the initial sparsification matching rather than an additional residual contribution.

Table 2 reports the average internal components of the modern estimator. For Erdős–Rényi graphs, the average sparsification matching size is 332.218, while the average Case 2 contribution $\widehat{|B_2|}/(kb)$ is 50.582. Nevertheless, the average value of μ_1 is 332.218, which is larger than the average value of μ_2, 245.191. Similarly, on near-regular graphs, μ_1 is dominated by $|M|$ and is larger than μ_2 on all trials. Power-law graphs are different: although $\widehat{|M'|}$ and $\widehat{|B_1|}$ are still zero on average, Case 2 is selected in a nontrivial fraction of trials.

Graph family	$\lvert M\rvert$	$\widehat{\lvert M'\rvert}$	$\widehat{\lvert B_1\rvert}$	$\widehat{\lvert B_2\rvert}/(kb)$	Mean μ_1	Mean μ_2
Erdős–Rényi	332.218	0.000	0.000	50.582	332.218	245.191
Power-law	130.448	0.000	0.000	49.100	130.448	125.514
Near-regular	369.060	0.000	0.000	20.549	369.060	236.739

Table 2: Average internal components of the modern estimator in the Tier 1 benchmark

Figure 5 visualizes the same decomposition by graph size. The dominant component is the sparsification matching $|M|$. The residual Case 1 terms are negligible in the tested graph families, while the scaled Case 2 term is visible but usually not large enough to make μ_2 exceed μ_1. This explains why Case 1 is often selected even though the additional residual Case 1 structure is inactive: the Case 1 formula includes the full sparsification matching $|M|$, whereas the Case 2 formula includes only $(1 - 1/b)|M|$ plus the scaled B_2 estimate.

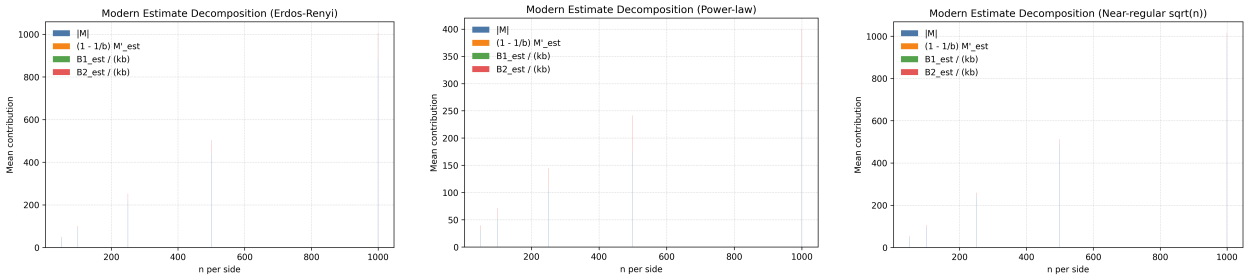

Figure 5: Decomposition of the modern estimator into the sparsification matching and scaled auxiliary estimates

Overall, the modern algorithm's empirical behavior on these graph families is driven primarily by the initial sparsification matching. The residual Case 1 estimates $\widehat{|M'|}$ and $\widehat{|B_1|}$ are inactive on average, while Case 2 contributes more visibly but usually does not overcome the larger coefficient on $|M|$ in μ_1. This helps explain the approximation-quality results: the modern estimator is competitive when the sparsification matching is already strong, but it does not obtain large additional gains from the residual Case 1 structure on these random graph families.